

Timer Lawn: Solving the Timer Wheel Overflow Problem for Large Scale and High Throughput Systems

Adam Lev-Libfeld

Tamar Labs
Tel-Aviv, Israel
adam@tamarlabs.com

Abstract—Timer facilities sit at the core of operating systems[1], network protocol implementations, real-time and stream processing, and several database capabilities. This paper presents Lawn, a field-tested timer data structure that maps queues by time-to-live (TTL) rather than by expiration time, so that its span is unbounded and it never rebuilds on overflow. We give the algorithm and its analysis, and evaluate an allocation-free implementation (Lawn2) against a production tickless hierarchical timing wheel and a naive single-level wheel. When the number of distinct TTLs is small relative to the number of live timers, Lawn2 deletes about 3.4 times faster, inserts about 1.4 to 1.9 times faster at scale, uses about 2.2 times less memory per timer, and holds memory flat across an unbounded TTL span. It also drains due timers faster than the wheel under every realistic TTL distribution we tested, by 4.9 to 27 times depending on how TTLs cluster, and keeps the idle per-tick check at $O(1)$ (about 1 ns versus the wheel’s cascade-bound $\sim 12.9\mu s$ at one million timers). These advantages are contingent: once the distinct-TTL count grows past roughly 1–5% of the live-timer count, per-tick cost becomes $O(t)$ and Lawn loses to the wheel. Even on the winning side of this boundary, “small” is relative rather than tiny in absolute terms: at one million live timers the crossover sits near nine thousand distinct TTLs, well within the range large-scale applications with a bounded, discrete TTL set actually use. We state this boundary quantitatively and report where Lawn should and should not be used.

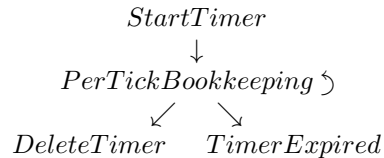
Keywords—Stream Processing, Timing Wheel, Dehydrator, Callout facilities, protocol implementations, Timers, Timer Facilities, Lawn.

I. Introduction

This paper presents a timer data structure designed for high-throughput computer systems, called Lawn. The current state-of-the-art algorithm, the Timing Wheel, is asymptotically optimal, but under some use cases (namely where the maximum TTL is unpredictable, or the needed tick resolution may change) it under-performs due to the overflow problem, which Lawn addresses. Lawn improves flexibility in TTL and tick resolution with no need for prior knowledge of the using system, under the single requirement that timer TTLs are drawn from a bounded set of discrete values. We make both a theoretical and an empirical case, and we are explicit about the regime in which the empirical advantage holds.

A. Model

In a similar manner to previous work[2], [3], [4], the model discussed in this paper consists of the following components, each corresponding with a different stage in the life cycle of a timer in the data store:



a) *StartTimer*(TTL,timerId,Payload): This routine is called by the client to start a timer that will expire after the TTL has passed. The client is also expected to supply a timer ID in order to distinguish it from other timers in the data store. Some implementations also allow the client to provide a Payload, usually some form of a callback action to be performed or data to be returned on timer expiration.

b) *PerTickBookkeeping*(): This routine encompasses all the actions, operations and callbacks to be performed as part of timer management and expiration check every interval as determined by the data store granularity. Upon discovery of an outstanding timer to expire *TimerExpired* is initiated by this routine.

c) *DeleteTimer*(timerId): The client may call this utility routine in order to remove from the data store an outstanding timer (corresponding with a given timer ID), before *PerTickBookkeeping* has marked it to be expired.

d) *TimerExpired*(timerId): Internally invoked by either *PerTickBookkeeping* or *DeleteTimer*, this routine entails all actions and operations needed to remove all traces of the timer corresponding with a given timer ID from the data store, and to invoke any callbacks that were provided as Payload during the *StartTimer* routine.

B. Callback run-time complexity

Since payload and callback run-time complexity varies between implementations, and the store of such data can be achieved in amortized $O(1)$ using a hash map with callback handling done in a discrete, highly (or even

embarrassingly) parallel manner, this paper disregards this aspect of timer stores.

II. Related Work

Timer implementations have been extensively studied, with a focus on efficiency, scalability, and correctness. This section reviews the key developments relevant to this work.

A. Timer Data Structures

The most influential work on timer data structures is the Timing Wheel algorithm proposed by Varghese and Lauck [5]. In the common case it provides $O(1)$ operations for timer insertion, deletion, and expiration, which is asymptotically optimal. However, as discussed below, it suffers from the overflow problem in scenarios with unpredictable TTL ranges.

Several extensions address the overflow problem. The Hierarchical Timing Wheel [2] uses multiple levels of wheels to handle a wider range of TTLs, at the cost of additional complexity, periodic cascade work, and memory overhead. The Multi-Pass approach [3] handles overflow by tracking the number of wheel cycles for each timer, but can lead to $O(n)$ complexity for PerTickBookkeeping in the worst case.

B. Concurrent Timer Implementations

Concurrent timer implementations are an active research area, driven by the need for scalable timer facilities in multi-threaded and distributed systems. [6] presents a scalable timer implementation for large-scale systems, while [7] proposes an auto-scaling timer implementation for cloud environments. Lock-free and RCU-based timer implementations have been proposed to minimize contention. We note that Lawn, as evaluated here, does not itself implement a lock-free timer. Concurrency is discussed as an architectural property and left as future work (Section IV-E).

C. Distributed Timer Implementations

The nature of the algorithms studied here lends itself to distribution to edge nodes in local applications, requiring no centralized timer management. Hence, distributed implementations are derived naively and are not discussed further.

III. Current Solutions

A. List and Tree Based Schemes

Operation	List Based	Tree Based
StartTimer	$O(n)$	$O(\log(n))$
PerTick	$O(1)$	$O(1)$
DeleteTimer	$O(n)$	$O(\log(n))$

TABLE I

Runtime Complexity for common data structure based schemes

Being included as an integral part of almost any modern programming language, these basic data structures enable

convenient addition of timer management to any software. That said, such simple structures are appropriate only for narrow use cases, where the number of timers is small or the ticks are far enough apart. The complexities above are the textbook figures for a sorted list and a balanced tree. They are not to be confused with the unsorted-append naive baseline benchmarked in Section VII, which is a different data structure.

B. Hashed Timing Wheel

Operation	Worst	Mean
StartTimer	$O(n)$	$O(1)$
PerTick	$O(n)$	$O(1)$
DeleteTimer	$O(1)$	$O(1)$

TABLE II

Runtime Complexity for the Hashed Timing Wheel scheme

The Hashed Timing Wheel was designed as an all-purpose timer storage solution for a unified system of known size and resolution[5], [2], [8], [6], [7]. In ideal conditions its run-time complexity is optimal, but real-world implementations are bound by the maximal TTL and resolution combination, or require a costly $O(n)$ run-time rebuild upon reaching such limits (in [3] this is referred to as “the overflow problem”). For large numbers of timers, the common workaround is to over-estimate the needed resolution and/or TTL so as to postpone rebuilding (resizing) for as long as possible.

C. Overflow Algorithms

To handle overflows correctly there are two valid options.

1) Increase current wheel slots (Alg. 1): Resize the overflowing wheel so it includes a larger number of slots into the future. This entails a worst-case cost of $O(n)$ upon any addition of an overflowing timer.

Algorithm 1 Timer Wheel Resize

```

1: function StartTimer(id, tll, payload)
2:   if tll > len(Wheel) then
3:     MissingSlots ← tll − len(Wheel)
4:     ExtWheel ← array of length len(Wheel) +
       MissingSlots
5:     for Slot, SlotIndex in Wheel do
6:       NewSlotIndex ← (len(Wheel) + SlotIndex)
       mod len(ExtWheel)
7:       ExtWheel[NewSlotIndex] ← Slot
8:     Wheel ← ExtWheel
9:     slot ← (current time + tll) mod len(Wheel)
10:    append payload to Wheel[slot]
11:    TimerHash[id] ← (payload, slot)

```

2) In-place addition of cycle count for each item (Alg. 2): Each element in a slot is stored with the number of wheel cycles needed before expiration. This entails a worst-case cost of $O(n)$ for the PerTickBookkeeping operation.

Algorithm 2 Timer Wheel Multi-Pass (in-place)

```

1: function StartTimer(id, ttl, payload)
2:    $T \leftarrow (\text{payload}=\text{payload}, \text{cycles}=(\text{current time}+t_{tl}) \div \text{len}(\text{Wheel}))$ 
3:    $\text{slot} \leftarrow (\text{current time}+t_{tl}) \bmod \text{len}(\text{Wheel})$ 
4:   append  $T$  to  $\text{Wheel}[\text{slot}]$ 
5:    $\text{TimerHash}[\text{id}] \leftarrow T$ 

6: function PerTickBookkeeping
7:   current slot  $\leftarrow$  current slot + 1
8:   for element in current slot do
9:     element.cycles  $\leftarrow$  element.cycles-1
10:    if element.cycles  $\leq$  0 then
11:      TimerExpired( $T_{id}$ )

```

IV. The Lawn Data Structure

A. Intended Use Cases

This algorithm was first developed during the construction of a large-scale, Stream Processing geographic intersection product[9] using a FastData[10] model. The data structure receives inputs from one or more systems that make use of a very limited range of TTLs in proportion to the number of concurrent timers they use.

a) Assumptions and Constraints: Lawn is intended for use under the assumption that

Unique TTL Count $\ll$ *Concurrent Timer Count*.

This is the load-bearing assumption of the whole approach. When it holds, TTL bucketing yields the results below. When it fails, Lawn degrades to $O(n)$ per tick and loses to the wheel. This assumption is expected to hold in most deployed systems: when TTLs originate from an algorithmic or configuration-driven source, such as a fixed backoff schedule or a small number of cache-tier classes, the set of possible values is bounded by construction and does not grow with the number of live timers. Section VII quantifies exactly where the crossover lies.

1) The Data Structure: Lawn is, at its core, a hash of sorted sets¹, much like the Timing Wheel. The main difference is the key used for hashing these sets: the timer TTL. Different timers are stored in the same set based only on their TTL, regardless of arrival time. Within each set, the timers are naturally sorted by time of arrival, effectively using the set as a queue (fig. 1). Because two timers in the same bucket share a TTL, any timer already in the queue will expire before a newly added one, so the queue stays sorted by expiration for free.

The data structure is analogous to blades of grass (hence the name): each blade grows from the roots up, and periodically (every *Tick*) the overgrown tops (the expired timers) are maintained by mowing the lawn to the desired level (current time).

¹These are the TTL ‘buckets’

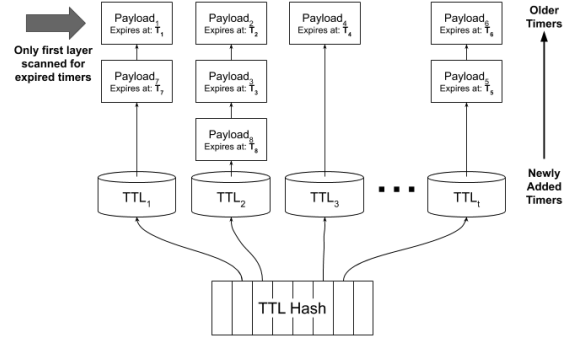


Fig. 1. A schematic view of the data structure components.

B. Algorithm

Note the *closest expiration* guard in PerTickBookkeeping: a tick on which nothing is due returns in $O(1)$ without scanning the buckets. This empty-tick fast path is important in practice, because most ticks expire nothing.

C. Two Implementations: Reference Lawn and Lawn2

The same algorithm admits two C implementations. They produce identical expiry schedules (verified by a differential test in Section VII) and differ only in who owns the timer node and how a timer is addressed for cancellation.

Reference Lawn favors readability and key addressing. The store owns each timer node, copies the client’s key, and indexes the timers in a hash map, so a timer can be cancelled by an arbitrary opaque key. That convenience costs one node allocation, one key copy, and two hash-map operations per insertion, which makes the reference implementation slower and heavier than a production timing wheel (Section VII). We present it as the direct expression of the algorithm.

Lawn2 keeps the algorithm but removes the per-operation allocation. Its nodes are intrusive and caller-owned (embedded in the client object), so an insertion neither allocates memory nor copies a key. Deletion is $O(1)$ through the node’s own links, which lets Lawn2 drop the element-id hash map, and the TTL→queue map uses open addressing with no per-entry allocation. The performance claims in this paper refer to Lawn2.

1) Correctness & Completeness: To prove correctness, we show that for each timer t with TTL t_{tl} , TimerExpired is called on t within one Tick of t_{tl} . Since each timer is stored exactly once in its corresponding bucket and buckets are independent, it suffices to demonstrate correctness within a bucket:

$$\forall T^{start}, T^{ttl} \in \mathbb{N} \quad \exists T^{stop} \in \mathbb{N} : T^{stop} - T^{start} \approx T^{ttl}$$

Alternatively, using the sorting analogy of G. Varghese et al.[2], given two triggers T_n, T_m :

Algorithm 3 The Lawn Data Store

Precondition:

```

1: id - a unique identifier of a timer.
2: tttl - a whole product of TickResolution representing
   the amount of time to wait before triggering the given
   timer payload action.
3: payload - the action to perform upon timer expiration.
4: current time - the local time of the system as a whole
   product of TickResolution

5: function InitLawn()
6:   TTLHash  $\leftarrow$  new empty hash set
7:   TimerHash  $\leftarrow$  new empty hash set
8:   closest expiration  $\leftarrow$  0

9: function StartTimer(id, tttl, payload)
10:  endtime  $\leftarrow$  current time + tttl
11:  T  $\leftarrow$  (endtime, tttl, id, payload)
12:  TimerHash[id]  $\leftarrow$  T
13:  if tttl  $\notin$  TTLHash then
14:    TTLHash[tttl]  $\leftarrow$  new empty queue
15:  TTLHash[tttl].insert(T)
16:  if endtime < closest expiration then
17:    closest expiration  $\leftarrow$  endtime

18: function PerTickBookkeeping()
19:  if current time < closest expiration then
20:    return
21:  for queue  $\in$  TTLHash do
22:    T  $\leftarrow$  peek(queue)
23:    while Tendtime < current time do
24:      TimerExpired(Tid)
25:      T  $\leftarrow$  peek(queue)
26:  if closest expiration = 0
   or Tendtime < closest expiration then
27:    closest expiration  $\leftarrow$  Tendtime

28: function TimerExpired(id)
29:  T  $\leftarrow$  TimerHash[id]
30:  DeleteTimer(T)
31:  do Tpayload

32: function DeleteTimer(id)
33:  T  $\leftarrow$  TimerHash[id]
34:  if Tendtime = closest expiration then
35:    closest expiration  $\leftarrow$  0
36:  TTLHash[Tttl].remove(T)
37:  TimerHash.remove(T)
38:  if TTLHash[Tttl] is empty then
39:    TTLHash.remove[tttl]

```

$$\forall T_n, T_m \mid T_n^{start} < T_m^{start}, T_n^{ttl} = T_m^{ttl} \Rightarrow T_n^{stop} < T_m^{stop}$$

Since each bucket contains only triggers with the same TTL, this simplifies to

$$\forall T_n, T_m \mid T_n^{start} < T_m^{start} \Rightarrow T_n^{stop} < T_m^{stop}$$

which, the bucket being a sorted set ordered by T^{start} and triggers being expired from old to new, is self-evident.

2) Formal Correctness Proof: Let T be a timer with TTL t_{ttl} started at time t_{start} . We prove T is expired at $t_{start} + t_{ttl}$. Let Q_{ttl} be the queue of timers with TTL t_{ttl} , $t_{current}$ the current time, and $t_{expiry} = t_{start} + t_{ttl}$.

Base Case: At t_{start} , T is inserted into Q_{ttl} with $t_{expiry} = t_{start} + t_{ttl}$.

Inductive Step: Assume at $t_{current} < t_{expiry}$ that T is in Q_{ttl} . If $t_{current} + 1 < t_{expiry}$, then T remains, as PerTickBookkeeping only expires timers with $t_{expiry} \leq t_{current} + 1$. If $t_{current} + 1 \geq t_{expiry}$, then T is expired. By induction, T is expired at t_{expiry} .

3) Complexity: The Lawn data structure is dense by design: every timer is stored exactly once, a new trigger adds at most one TTL bucket, and empty buckets are removed. The footprint therefore grows linearly with the number of timers, so space is $O(n)$ (confirmed empirically in Section VII).

Operation	Worst	Mean
StartTimer	$O(1)$	$O(1)$
PerTick	$O(n)$	$O(t \sim 1)$
DeleteTimer	$O(1)$	$O(1)$

TABLE III

Runtime Complexity for the Lawn scheme

StartTimer and DeleteTimer are $O(1)$ amortized. The worst case is $O(n)$ on a hash-map/table resize, as is standard for hash-based structures (visible in the tail of the measured insert latency). An empty tick is $O(1)$ via the *closest expiration* guard. A tick that does expire timers scans the head of each of the t live TTL buckets, so its cost is $O(t + k)$ for k expired timers, with a worst case of $O(n)$ (all timers due in one tick). When t is bounded (i.e. $t = O(1)$, the assumption of Section 4.A), the per-tick cost is $O(1)$. Otherwise it is $O(t)$ and Lawn is worse than a wheel. This dependence on t is the algorithm's defining trade-off and is measured directly in Section VII.

a) Space: is $O(n)$: at worst each timer occupies its own bucket plus a single hash entry. Theoretically this footprint is bounded above by that of the Hashed Hierarchical Timing Wheel, whose multi-level structure can point at a single timer through a chain of wheels. We did not measure a hierarchical wheel directly. Against the single/tickless wheel we benchmarked, Lawn2 uses less memory but the reference Lawn uses more (Section VII).

4) Queueing Theory Sketch: Modeling each TTL bucket as an M/M/1 queue with Poisson arrivals (rate λ_{ttl}) and a single server (PerTickBookkeeping) gives the standard $E[N_{ttl}] = \lambda_{ttl} / (\mu_{ttl} - \lambda_{ttl})$ and $E[W_{ttl}] = 1 / (\mu_{ttl} -$

λ_{ttl}), with a batch-processing correction $E[W_{ttl}] = B/(2\mu_{ttl}) + 1/(\mu_{ttl} - \lambda_{ttl})$. We stress that M/M/1 assumes memoryless (exponential) service, whereas a Lawn timer’s expiry is deterministic (a fixed discrete TTL), so a deterministic-service M/D/1 model is a better fit and yields smaller waiting times. These bounds are modeling sketches and are not empirically validated here. The system is stable when $\lambda_{ttl} < \mu_{ttl}$ for every bucket. The $O(n)$ space bound, by contrast, is measured (Section VII).

D. Memory Access Patterns

Lawn organizes timers by TTL value rather than expiration time, creating a natural grouping of timers with the same TTL. Each bucket is a queue, giving sequential access when a bucket is drained. The timer index provides $O(1)$ lookups at the cost of one indirection, a common pattern also used by the Hashed Hierarchical Timing Wheel[2]. We did not instrument cache behavior with hardware counters, so we make no measured cache-locality or fragmentation claim. The memory results in Section VII are footprint measurements only. Lawn2 further reduces overhead by embedding nodes in client objects (object pooling by construction) and by avoiding per-entry allocation in its TTL table.

E. Concurrency Considerations

Because TTL buckets are independent, a Lawn can be partitioned across workers, threads, or machines by bucket, and a partitioned deployment needs no synchronization across buckets. We evaluate this honestly in Section VII: on a single multi-core machine, wrapping each partition in its own lock, Lawn2 keeps a modest throughput edge over the wheel but no implementation scales up with thread count under coarse locking. We did not implement a lock-free or per-bucket-lock timer. A genuinely concurrent Lawn is left as future work, and we make no lock-free superiority claim.

V. Comparison and Reflection

General-purpose systems, or applications with highly predictable needs, may prefer the stable run-time of the Timing Wheel (which is asymptotically optimal). Large-scale serving systems, however, suffer from the overflow problem under unpredictable usage. Lawn trades a dependence on the distinct-TTL count for an unbounded span with no rebuilds. It is competitive with, and often faster than, the state of the art only within its assumed regime of low TTL variance (Section VII). It is not a universal improvement.

VI. Limitations

Lawn’s advantages are conditional, so understanding the limits is essential to choosing it.

Operation	TW (Resize)	TW (Multi-Pass)	Lawn
StartTimer	$O(n)^*$	$O(1)$	$O(1)^\dagger$
PerTick	$O(1)$	$O(n)^\ddagger$	$O(t)^\S$
DeleteTimer	$O(1)$	$O(1)$	$O(1)^\dagger$
TimerExpired	$O(1)$	$O(1)$	$O(1)$
space	$O(n)$	$O(n)$	$O(n)$

* on overflow, otherwise amortized $O(1)$. † amortized, worst case $O(n)$ on a resize. ‡ worst case, from the multi-pass cascade. § $O(1)$ empty tick and mean when $t = O(1)$, $O(n)$ worst case.

TABLE IV

Runtime complexity comparison. The Multi-Pass $O(n)$ penalty falls on PerTick (the cascade), consistent with Alg. 2.

A. TTL Distribution Sensitivity

Lawn’s performance depends directly on the distribution of TTL values. When TTLs are drawn from a continuous or highly variable distribution, the number of buckets t grows toward the number of timers n , and PerTickBookkeeping approaches $O(n)$. Section VII measures the crossover: Lawn2 loses to the wheel once the distinct-TTL fraction t/N exceeds roughly 0.9–4.6% (tightening as N grows), and is up to two orders of magnitude slower when nearly every timer has a distinct TTL.

B. Memory Overhead for Small Timer Counts

At small scale, Lawn2’s fixed per-structure overhead dominates: at one thousand timers it uses more memory per timer than the wheel, and the advantage appears only at scale (Section VII).

C. Time Granularity and Implementation Complexity

Hierarchical wheels handle variable tick granularity more naturally, level by level, which may suit applications needing very fine-grained precision. Lawn’s mixture of a hash table and per-bucket queues is simple in concept but requires care to implement efficiently (dense layout, avoiding cache misses), especially in a distributed setting.

VII. Experimental Evaluation

We validate the analysis and compare Lawn against timing-wheel baselines.

A. Methodology

We benchmark four C implementations behind one interface: the reference Lawn (src/lawn.c), the optimized Lawn2 (src/lawn2.c), Timer Wheel (William Ahern’s tickless hierarchical wheel [8], used as the production baseline), and a Naive single-level growing ring. All run on an integer logical clock, so timing is deterministic and wall-clock-free. A differential correctness gate is run first: all four implementations must produce byte-identical per-tick expiry schedules (with and without deletions, and for large TTLs that force cascades) before any number is trusted. They do.

Each operation is timed with the finest monotonic clock available (CLOCK_UPTIME_RAW, ~41 ns quantized on the test machine), so sub-40 ns operations are timed

in micro-batches of 256 to lift them off the clock floor. Measurements use fixed seeds (base 1234) with 7 runs and 2 warmup runs (3/1 at the largest scale). The C results below are authoritative for absolute cost and cross-implementation ranking. We also maintain a Python suite. It is useful for confirming scaling shape, but its absolute numbers carry interpreter overhead and invert the insert ranking, so we do not cite Python numbers here. Hardware: Apple M4 Pro (12 cores, 8 performance + 4 efficiency), macOS 26.3, Apple clang 21.

B. Basic Operations

We evaluate insertion, deletion, and the two tick paths across scales of one thousand to one million timers.

Naive wins several of the metrics below, which is worth explaining once before the operation-by-operation results, rather than as a series of isolated asides. Naive is a single-level ring sized to the largest TTL it currently holds: while that span stays small, insertion, an empty tick, and expiry all resolve to one direct array index, with no bucket scan and no cascade. Every benchmark in this subsection holds the span small by construction: the operation baselines fix a TTL span of 2560 ticks, and the expiry test narrows it further to a 200-tick window. Under that condition naive has no bucket-scan or cascade cost to pay at all, so it is not merely competitive, it is the fastest structure measured.

That condition is exactly what Figure 6 varies, and it is exactly where naive fails. Once the TTL span is not known in advance and grows, naive’s ring must grow to cover it, which means rehashing every live timer into the larger ring: an $O(n)$ cost of the same kind as the wheel-resize overflow algorithm (Algorithm 1). The tickless wheel avoids that rehash by paying a bounded cascade cost instead, visible in Figure 10 as the wheel’s own cost growing with scale, and Lawn/Lawn2 avoid it entirely by bucketing on TTL value rather than absolute time. Naive is included as the textbook overflow victim, not as a production candidate: unlike the tickless wheel, a deployed library used as the production baseline throughout this paper, naive exists specifically to make the overflow problem visible. Read the results below accordingly: naive wins wherever the TTL span is small and bounded in advance, and loses precisely when that assumption breaks, which is the condition this paper is about.

1) Insertion: Figure 2 shows insertion latency. Lawn2 is comparable to the wheel at ten thousand timers and about 1.4 to 1.9 times faster at scale (5.8ns vs. 8.5ns at one million, and 4.1ns vs. 7.9ns at one hundred thousand). The unsorted naive ring is fastest of all (~ 2.4 ns), and the reference Lawn is slower (~ 88 ns at one million, from its per-insert allocation and key copy), which is why the performance claims use Lawn2.

2) Deletion: Figure 3 shows deletion latency. Lawn2 is consistently about 3.1 to 3.6 times faster than the wheel (40.7ns vs. 127.7ns at one million, and 4.9ns vs. 17.3ns

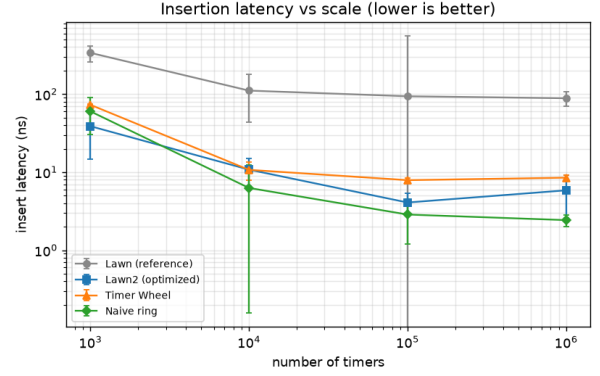


Fig. 2. Insertion latency across scales (C, ns).

at one hundred thousand), the ratio roughly constant so the absolute gap grows with scale.

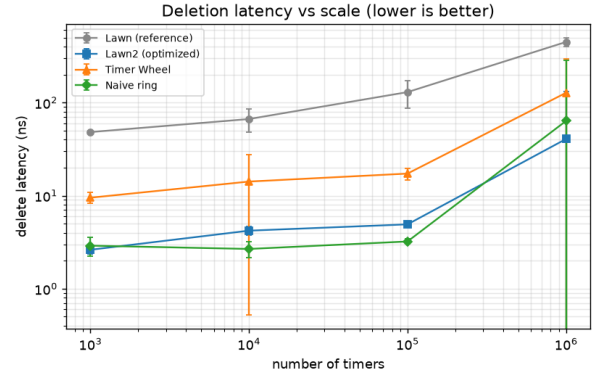


Fig. 3. Deletion latency across scales (C, ns).

3) Expiry cost: Figure 4 shows the cost of a tick that drains due timers, the operation that does the actual work of firing callbacks, using a fixed 200-tick window so every timer is due within it regardless of scale. Lawn2 is about 2.2 times faster than the wheel at one million timers (408 μ s vs. 884 μ s): both must inspect every live TTL bucket or wheel level on a draining tick, and Lawn2 does so more cheaply. The naive ring pays no such inspection: it drains its due slot by direct index and so dominates both here (~ 6.3 μ s), for the reason given above.

4) Empty-tick (bookkeeping) cost: A tick on which nothing is due does no useful work by itself, but its cost still matters because it runs on every clock tick regardless of load, so a per-tick tax here is paid continuously rather than only when timers actually fire. Lawn2 is flat at $O(1)$ (~ 1 ns) at every scale, while the tickless wheel pays a cascade cost that grows to ~ 12.9 μ s at one million timers (with high variance), more than four orders of magnitude higher. This is the empirical form of the overflow problem: the wheel does work proportional to elapsed time and level structure, Lawn does not. The same gap, and its continuation far past one million timers, is visualized

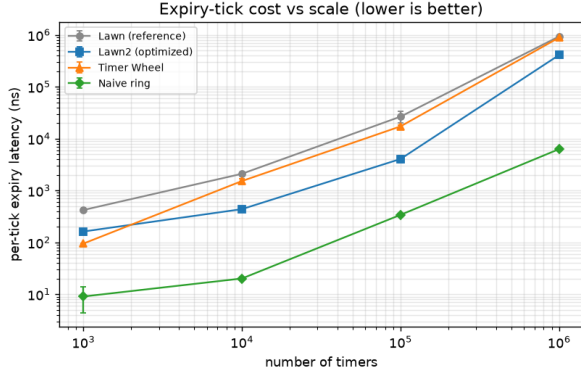


Fig. 4. Per-tick expiry latency across scales (C, ns).

directly in Figures 10 and 12 in Section VII-F: Lawn2 stays at ~ 1 ns independent of t (the *closest expiration guard*) at every scale tested, while the wheel’s cascade cost is unrelated to t and grows with N .

C. Memory Usage

Figure 5 shows per-timer memory at a fixed, modest TTL span of 2560 ticks. At scale Lawn2 uses about 40 bytes per timer versus the wheel’s about 88 (2.2 times less), and grows more slowly. At one thousand timers Lawn2 is heavier (fixed overhead). The reference Lawn uses more than the wheel (~ 113 bytes) at scale. The naive ring is lightest of all here (~ 38 bytes per timer): its cost at this fixed span is just per-timer bookkeeping plus an amortized, doubling array. Figure 6 shows what happens once the span is not fixed: total memory against TTL span growing a thousandfold, at both the main one hundred thousand-timer baseline and the ten million-timer baseline from Section VII-F. Lawn, Lawn2, and the tickless wheel are flat at both baselines, while the naive single-level ring grows, the overflow rehash described above, 4.2 MB to 12.2 MB at one hundred thousand timers and 507.5 MB to 513.8 MB at ten million. Section VII-F shows this same growth normalized to each baseline’s own starting point (Figure 11): in relative terms the overflow effect is far weaker at the larger baseline, 2.9 times growth versus 1.2%.

D. Workload Characterization

We measure per-tick expiry cost at one hundred thousand timers under different TTL distributions (Lawn2 vs. the wheel):

- Single TTL (all timers share one TTL): 429 ns vs. 11,552 ns ($\sim 27\times$).
- Few TTLs (ten distinct values): 2,780 ns vs. 17,206 ns ($\sim 6.2\times$).
- Bursty arrivals: 1,482 ns vs. 11,384 ns ($\sim 7.7\times$).
- Uniform: 3,770 ns vs. 18,434 ns ($\sim 4.9\times$).

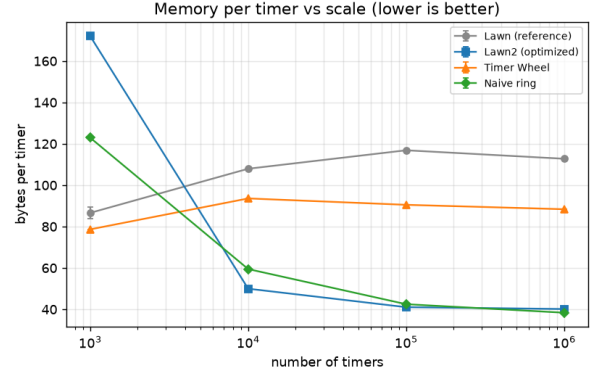


Fig. 5. Memory per timer across scales (C, bytes).

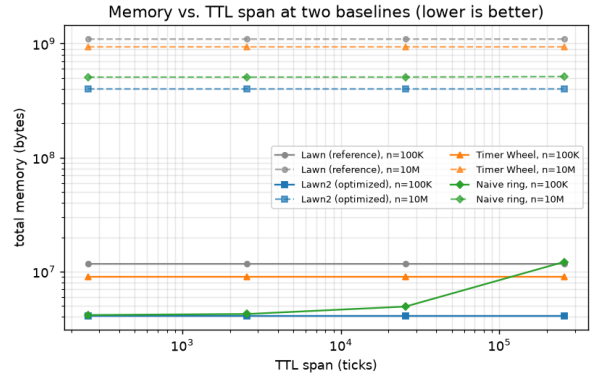


Fig. 6. Total memory vs. TTL span at two baselines, one hundred thousand and ten million timers (C, bytes, lower is better). The naive ring overflows at both while Lawn, Lawn2, and the wheel stay flat.

Lawn2 is faster on expiry across all of these. The advantage is largest when TTLs cluster, as expected. Figure 7 shows the same comparison directly.

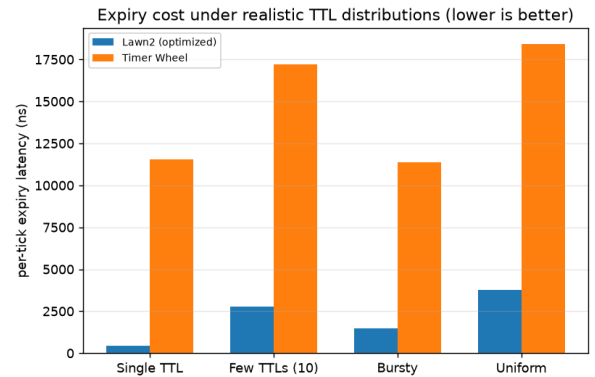


Fig. 7. Per-tick expiry latency, Lawn2 vs. the wheel, under four realistic TTL distributions (C, ns, lower is better).

E. The Distinct-TTL / Timer-Count Crossover

The central caveat of the paper is measured directly in Figure 8, which plots Lawn2’s full-lifecycle speedup

over the wheel (Wheel cost divided by Lawn2 cost) as the distinct-TTL fraction t/N varies. Lawn2 wins (speedup above one) while t/N is small, and crosses over to losing at $t/N \approx 4.6\%$ ($N = 10^4$), 1.5% ($N = 10^5$), and 0.9% ($N = 10^6$). The boundary tightens as N grows. When nearly every timer has a distinct TTL ($t/N = 1$), Lawn2 is up to about 349 times slower than the wheel. This is the honest boundary of applicability: Lawn is a win precisely when distinct TTLs stay well under a few percent of live timers. In absolute terms this is more permissive than the percentage suggests: at $N = 10^4$ the boundary is about 460 distinct TTLs, at $N = 10^5$ about 1500, and at $N = 10^6$ about 9000, comfortably within the range of a configuration-driven or otherwise bounded TTL set at large scale rather than a handful of values. We additionally swept $N = 10^7$ across the same distinct-TTL range. Lawn2 wins throughout it, but that range only reaches $t/N = 0.1\%$, below where the tightening trend above predicts the $N = 10^7$ crossover actually falls, so this point confirms the low- t/N region holds at larger N without locating where the boundary now sits. We flag this as a gap rather than claim the crossover has vanished at this scale. Section VII-F finds a second, independent failure mode of the wheel at far larger N , driven by absolute timer count rather than distinct-TTL count, which we defer there to keep this boundary specific to the axis it measures.

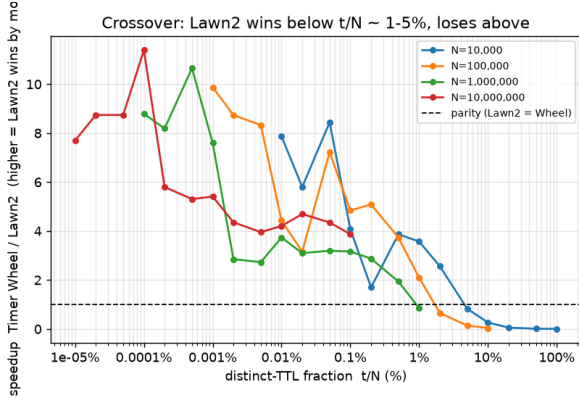


Fig. 8. Lifecycle speedup of Lawn2 over the Timer Wheel vs. the distinct-TTL fraction t/N (C). Higher means Lawn2 wins by more.

F. Robustness at Extended Scale

The results above cover one thousand to one million timers, matching the paper’s main methodology. To check that the core claims are not artifacts of that range, we separately swept timer counts far beyond it, at a fixed distinct-TTL count of ten thousand (denser than the main baseline of one hundred), stopping each implementation at the largest count that fit in memory on the 24GB test machine: Lawn2 to 150 million, the naive ring to 190 million, the tickless wheel to 118 million, and reference Lawn to 100 million. This is a single-machine check, not a

repeated statistical study, so we report it separately from the main results and state only what is unambiguous in the data.

Lawn2’s insert, empty-tick, and memory costs stay flat across the entire extended range with no exception. Insert is 6.7 ns at one million timers and 6.5 ns at 150 million. The empty tick is 1.8 ns at one million and 1.5 ns at 150 million. Memory is 40.7 B/timer at one million, converging to 40.0 B/timer by ten million and holding there, with zero measured variance, through 150 million. Figure 9 shows this holds for every implementation tested, not just Lawn2: per-timer memory is flat out to hundreds of millions of timers across the board, in contrast to the scale-driven empty-tick cost below, which is specific to the wheel. The tickless wheel’s empty-tick cost, in contrast, keeps growing with absolute timer count well past the one-million-timer range covered above: $8.5 \mu\text{s}$ at one million, $87.9 \mu\text{s}$ at ten million, $429.6 \mu\text{s}$ at one hundred million, and $510.6 \mu\text{s}$ at 118 million. The crossover documented in Figure 8 is therefore not a small-scale artifact: the wheel’s cost continues to grow with absolute timer count at every scale we could test.

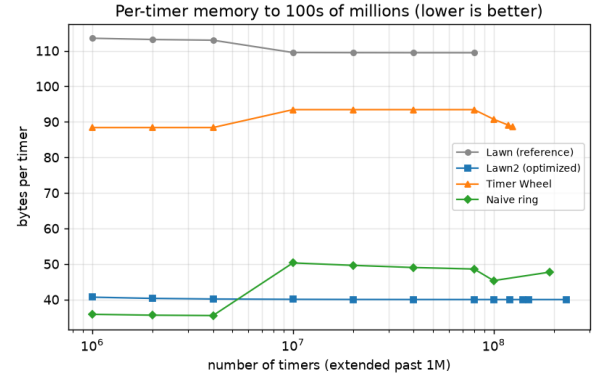


Fig. 9. Per-timer memory vs. timer count, extended to hundreds of millions (C, bytes, lower is better). Flat for every implementation.

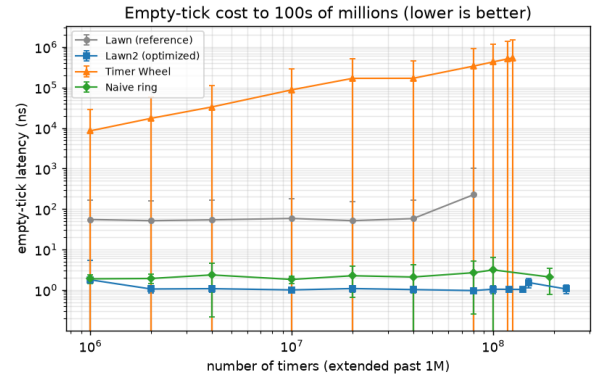


Fig. 10. Empty-tick latency vs. timer count, extended to hundreds of millions (C, ns). Lawn2 stays flat while the wheel’s cost keeps climbing.

The naive ring’s expiry advantage over Lawn2, conversely, narrows as scale grows and then holds steady rather than closing further: 43.4 times faster at one million timers, 28.0 times at ten million, 22.5 times at 40 million, and 21.3 times at one hundred million. One caveat applies across every implementation at this extended scale: deletion latency does not follow a clean trend for any of them, varying by 5 to 20 times between adjacent measured scales with no consistent direction. We attribute this to the interaction between a fully randomized deletion order and cache and allocator state on a very large working set, not to a change in the underlying algorithm, since the same non-monotonic pattern appears in all four implementations rather than only the ones with a case for concern.

Beyond varying timer count alone, we reran the complete parameter sweep, distinct-TTL count, TTL span, and workload, at a fixed baseline of ten million timers instead of the main results’ one hundred thousand, to check whether the crossover and overflow effects documented above scale in the simple way one might expect. They do not, and they diverge in opposite directions. The overflow effect on the naive ring’s memory weakens sharply in relative terms at this baseline. At the original baseline its memory grows 2.9 times across the TTL-span sweep, 4.16 to 12.16 MB. At ten million timers the identical sweep produces only 1.2% growth, 507.5 to 513.8 MB, because the per-timer cost that dominates total memory at this scale swamps the fixed ring-overhead cost that dominated it at the smaller scale. Figure 11 shows both curves normalized to their own smallest-span point, making the flattening directly visible. This sweep measures steady-state footprint, not the cost of an actual resize event, which moves proportionally more data at larger N and so plausibly grows worse, not better, even as the steady-state tax shrinks. We did not measure a live resize event directly and flag this as a gap rather than a resolved question.

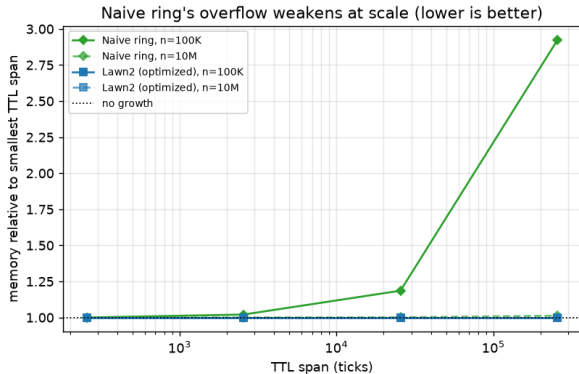


Fig. 11. Naive ring’s memory growth across the TTL-span sweep at two baselines, normalized to each series’ own smallest-span point (C, lower is better). The 2.9x growth at one hundred thousand timers falls to 1.2% at ten million. Lawn2 is flat at both baselines.

The distinct-TTL crossover is, conversely, partly super-

sed by a second and more severe effect at this scale. At the original baseline the wheel’s empty-tick cost across the distinct-TTL sweep is a mild 270 to 522 ns, sensitive mainly to how many distinct TTLs are in use. At ten million timers, under the identical sweep, the wheel’s cost balloons to 24 to 145 μ s and no longer tracks distinct-TTL count in any clean, monotonic way. Lawn2 is unmoved by either axis, holding at 1.0 to 1.1 ns regardless of distinct-TTL count or absolute scale: its advantage does not depend on which of the two effects, TTL diversity or the wheel’s own scale-driven cost, happens to dominate.

The obvious candidate explanation for the wheel’s cost growing with N at fixed TTL span is density, the number of live timers sharing each tick, N divided by the TTL span. We tested this directly by sweeping the TTL span itself at both baselines and converting to density (Figure 12), rather than inferring it from the distinct-TTL sweep above. Density alone does not explain the effect. One hundred thousand timers over a span of 2560 ticks and ten million timers over a span of 256 000 ticks both give a density of about 39 timers per tick, but the wheel’s cost differs by about 128 times between them, 272.9 ns versus 35 107.3 ns. Absolute timer count, not density, is the dominant driver of this cost. Lawn2 is unaffected by either baseline or density, holding at about 1 ns throughout, the same flat behavior seen against the distinct-TTL sweep above. We did not isolate the specific mechanism behind the wheel’s cost, larger absolute cascade populations, cache locality at a larger working set, or some combination, and flag this as an open question rather than speculate beyond the measurement.

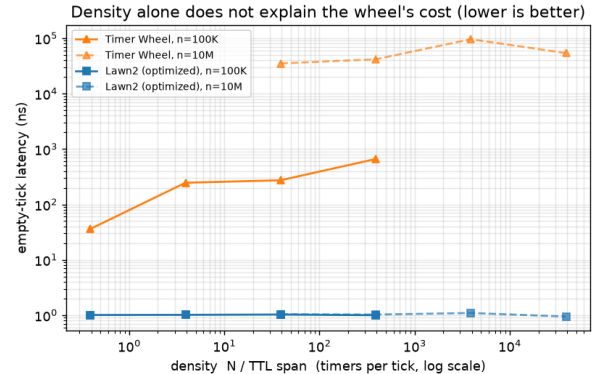


Fig. 12. Empty-tick latency vs. density (timers per tick, N / TTL span) at two baselines (C, lower is better). Matched density does not equalize the wheel’s cost between baselines, so density alone does not explain its blowup. Lawn2 is flat throughout.

G. Concurrency

Figure 13 reports single-machine throughput under an external per-shard mutex applied identically to every implementation. Single-threaded, Lawn2 reaches 121 M ops/s versus the wheel’s 90 M (the naive ring leads at 134 M). As threads are added under coarse locking,

aggregate throughput falls for every implementation on this machine (turbo and core-asymmetry effects). Lawn2 retains only a modest ~ 1.3 to $1.4\times$ edge at twelve threads (18.8M vs. 14.1M ops/s). We found no evidence of the large concurrency advantages sometimes attributed to bucketed timers. Realizing one would require a lock-free implementation, which we leave to future work.

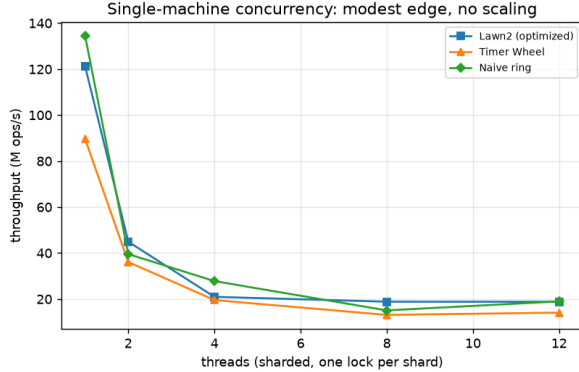


Fig. 13. Single-machine throughput vs. thread count, sharded (C).

VIII. Conclusion

Lawn is a simple, overflow-free timer data structure that maps queues by TTL rather than by expiration time, giving an unbounded span with no rebuilds. Its optimized implementation, Lawn2, beats a production tickless hierarchical timing wheel on deletion ($\sim 3.4\times$), on insertion at scale (~ 1.4 to $1.9\times$), on per-timer memory at scale ($\sim 2.2\times$), and on draining due timers under every realistic TTL distribution tested (~ 4.9 to $27\times$), while keeping memory flat over an unbounded TTL span. The same $O(1)$ bucketing also keeps the idle per-tick check constant, against the wheel’s cascade-bound cost that grows with scale. These gains are real but conditional: they hold only while the number of distinct TTLs stays under roughly one to five percent of the live-timer count, and Lawn degrades to $O(n)$ per tick and loses to the wheel beyond that. Lawn is therefore the right choice for large-scale systems whose timers are drawn from a bounded set of discrete TTLs, and the wrong one for continuous or near-unique TTL distributions.

A. Contributions

- Data structure: the TTL-keyed Queue-Map (Lawn) that removes the overflow problem, plus an allocation-free implementation (Lawn2).
- Analysis: correctness, and complexity with an explicit, measured account of the $O(t)$ per-tick dependence and the empty-tick $O(1)$ fast path.
- Honest evaluation: a differential-gated C benchmark of four implementations across scale, TTL span, distinct-TTL count, and workload, quantifying both where Lawn wins and the t/N crossover where it loses.

- Deployment: Lawn is in production use by several organizations, some independent of the author, including Redis Internals for streams and other internal timers[11], the ReDe Redis dehydration module[12], Mellanox RDMA timers for Unified Communication X[13], client-device rate-limiting timers[9], and an independent *clib* timer-management utility library, with the source available for most of these.

B. Future Work

- Lock-free / per-bucket-lock Lawn: a genuinely concurrent implementation, and an evaluation on NUMA hardware (not attempted here).
- Formal verification of correctness using theorem provers or model checkers.
- Deterministic-service queueing model (M/D/1) with empirical validation of the latency bounds.
- Distributed implementations for geographically distributed systems.

References

- [1] A. Costello, A. M. Costello, G. Varghese, and G. Varghese, “Redesigning the bsd callout and timer facilities,” tech. rep., Washington University in St Louis, 1995.
- [2] G. Varghese and A. Lauck, “Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility,” IEEE/ACM transactions on networking, vol. 5, pp. 824–834, December 1997.
- [3] D. Sleator and R. Brown, “Calendar queues: A fast $o(1)$ priority queue implementation for the simulation event set problem,” Communications of the ACM, vol. 31, Oct. 1988.
- [4] D. W. Jones, “An empirical comparison of priority-queue and event-set implementations,” Commun. ACM, vol. 29, pp. 300–311, Apr. 1986.
- [5] G. Varghese and T. Lauck, “Hashed and hierarchical timing wheels: Data structures for the efficient implementation of a timer facility,” SIGOPS Oper. Syst. Rev., vol. 21, pp. 25–38, Nov. 1987.
- [6] H. Hu, “Large-scale timer management.” US Patent no. US8307030B1.
- [7] Z. ZhouIvan, “On-demand scalable timer wheel.” US Patent no. US20140298073A1.
- [8] N. Welch, “timeout.c: Tickless hierarchical timing wheel implementation.” <http://25thandclement.com/~william/projects/timeout.c.html>.
- [9] A. M. Adam Matan, Adam Lev-Libfeld, “Vioozer geo-tagging system.” <http://www.vioozer.com/>, 2017.
- [10] A. Lev-Libfeld and A. Margolin, “Fast data - moving beyond from big data’s map-reduce,” GeoPython, vol. 1, pp. 3–6, June 2016.
- [11] S. Ibarra, “Redis.” <https://redis.io/>, 2009.
- [12] A. Lev-Libfeld, “A redis element dehydration module.” www.tamarlabs.com/ReDe/, 2017.
- [13] “Unified communication x.” <https://github.com/openucx/ucx/tree/master/src/ucs/time>, 2018.